

SQL Fundamentals

Database Management

| Lecture Notes

INSERT

SELECT

DESCRIBE

% Operator

Table of Contents

01	INSERT Statement	Adding records into a table
02	SELECT Statement	Querying and retrieving data
03	DESCRIBE / DESC	Viewing table structure
04	% (LIKE) Operator	Pattern matching in queries
05	Combined Examples	Real-world SQL queries

01 INSERT STATEMENT

The INSERT INTO statement is used to add new rows (records) into a database table.

What it does

Inserts one or more new rows
into an existing SQL table.
Data must match column types.

When to use it

When entering new records,
importing bulk data, or
populating tables initially.

Syntax

```
-- Syntax 1: Insert with all columns
INSERT INTO table_name
VALUES (value1, value2, value3, ...);

-- Syntax 2: Insert with specific columns
INSERT INTO table_name (col1, col2, col3)
VALUES (value1, value2, value3);
```

Example — Students Table

```
-- Create table first
CREATE TABLE Students (
    StudentID INT,
    Name      VARCHAR(50),
    Age       INT,
    City      VARCHAR(50)
);

-- Insert a single record
INSERT INTO Students (StudentID, Name, Age, City)
VALUES (101, 'Riya', 20, 'Pune');

-- Insert multiple records
INSERT INTO Students VALUES
    (102, 'Aarav', 21, 'Mumbai'),
    (103, 'Sneha', 22, 'Delhi');
```

Key Points

- String values must be wrapped in single quotes: 'Riya'
- Column order in INSERT must match the VALUES order.
- If inserting all columns, column names can be omitted.
- NULL can be used for optional / missing values.

02 SELECT STATEMENT

The SELECT statement retrieves data from one or more tables. It is the most used SQL command.

SELECT *

Fetch ALL columns
from a table

SELECT col

Fetch SPECIFIC
columns only

WHERE clause

Filter rows by
a condition

Basic Syntax

```
-- Select all columns
SELECT * FROM table_name;

-- Select specific columns
SELECT column1, column2 FROM table_name;

-- Select with condition (WHERE)
SELECT * FROM table_name WHERE condition;
```

Examples — Using Students Table

```
-- Fetch all students
SELECT * FROM Students;

-- Fetch only Name and City
SELECT Name, City FROM Students;

-- Students from Mumbai
SELECT * FROM Students WHERE City = 'Mumbai';

-- Students older than 20
SELECT * FROM Students WHERE Age > 20;

-- Multiple conditions
SELECT * FROM Students WHERE Age >= 20 AND City = 'Pune';
```

Useful SELECT Clauses

ORDER BY

Sort results: `SELECT * FROM Students ORDER BY Age DESC;`

LIMIT

Limit rows: `SELECT * FROM Students LIMIT 5;`

DISTINCT

Unique values: `SELECT DISTINCT City FROM Students;`

AS

Aliases: `SELECT Name AS StudentName FROM Students;`

03 DESCRIBE / DESC

DESCRIBE (or DESC) shows the structure of a table — column names, data types, and constraints.

What it shows

Column Name
Data Type (INT, VARCHAR...)
NULL / NOT NULL, Key, Default

Why it's useful

Check table design before writing queries. Understand field types and constraints.

Syntax

```
DESCRIBE table_name; -- full keyword
DESC table_name;     -- shorthand
```

Example

```
DESC Students;
```

Sample Output:

Field	Type	Null	Key	Default	Extra
StudentID	INT(11)	NO	PRI	NULL	
Name	VARCHAR(50)	YES		NULL	
Age	INT(11)	YES		NULL	
City	VARCHAR(50)	YES		NULL	

Key Columns Explained

Field	Name of the column
Type	Data type — INT, VARCHAR, DATE, FLOAT, etc.
Null	YES = can be empty, NO = must have a value
Key	PRI = Primary Key, UNI = Unique, MUL = Foreign Key
Default	Default value if none is provided during INSERT

04 % (LIKE) OPERATOR

% Wildcard — Pattern Matching in SQL

The % operator is used with the LIKE keyword to search for a pattern within a column's values.

%ab**Ends with 'ab'**

'%Kumar' matches Raj Kumar, Suresh Kumar

abc%**Starts with 'abc'**

'R%' matches Riya, Raj, Rohit

%abc%**Contains 'abc'**

'%ya%' matches Riya, Ayan, Maya

Syntax with LIKE

```
SELECT * FROM table_name
WHERE column_name LIKE 'pattern';
```

Practical Examples

```
-- Names starting with 'R'
SELECT * FROM Students WHERE Name LIKE 'R%';

-- Names ending with 'a'
SELECT * FROM Students WHERE Name LIKE '%a';

-- Names containing 'ya'
SELECT * FROM Students WHERE Name LIKE '%ya%';

-- Cities starting with 'M' and ending with 'i'
SELECT * FROM Students WHERE City LIKE 'M%i';

-- NOT LIKE: exclude pattern
SELECT * FROM Students WHERE City NOT LIKE '%Delhi%';
```

Bonus: _ (Underscore) Wildcard

_ matches exactly ONE character. Example: WHERE Name LIKE '_iya' matches 'Riya', 'Tiya'

05 COMBINED EXAMPLES

```
-- Step 1: Create table
CREATE TABLE Employees (
    EmpID    INT,
    Name     VARCHAR(60),
    Dept     VARCHAR(40),
    Salary   FLOAT
);

-- Step 2: Insert records
INSERT INTO Employees VALUES (1, 'Riya Kaldate', 'IT', 55000);
INSERT INTO Employees VALUES (2, 'Aarav Shah', 'Finance', 48000);
INSERT INTO Employees VALUES (3, 'Sneha Patil', 'IT', 62000);
INSERT INTO Employees VALUES (4, 'Rahul Sharma', 'HR', 40000);

-- Step 3: View structure
DESC Employees;

-- Step 4: Select all
SELECT * FROM Employees;

-- Step 5: Select IT department employees
SELECT Name, Salary FROM Employees WHERE Dept = 'IT';

-- Step 6: Use % to find names containing 'a'
SELECT * FROM Employees WHERE Name LIKE '%a%';

-- Step 7: Names starting with 'R'
SELECT * FROM Employees WHERE Name LIKE 'R%';
```

Quick Revision Summary

INSERT

Adds new rows to a table. Use VALUES to provide data.

SELECT

Retrieves data. Use * for all cols or list specific ones.

DESCRIBE

Shows column names, types, and constraints of a table.

% (LIKE)

Wildcard for pattern search. % = any chars, _ = one char.